ITNPAI2: AI for Computer Vision

Ariadne

A Synthetic Proof of Concept for Air-to-Ground Optical Navigation

Student ID: 3539054

April 1, 2026

Abstract

Modern search and rescue (SAR) and tactical robots face serious challenges when GPS signals are unavailable. When sensors stop working or satellite signals are blocked, ground vehicles lose their ability to navigate effectively. To solve this issue, this paper presents *Ariadne*, an autonomous system that helps guide ground vehicles using aerial imaging. This system uses a single high-altitude camera mounted on a drone (UAV) to direct an unmanned ground vehicle (UGV). The system is built on a few key components: it starts with a YOLO-based module to identify targets, followed by a U-Net neural network to segment the environment, and finishes with an A^* algorithm to plan a safe path. To test how well this system works without the stochastic noise of real-world environments, it was assessed using a custom dataset of artificially created 2D mazes. The results showed that this method can successfully turn aerial image data into safe and efficient routes for ground vehicles. This proof of concept demonstrates a solid and scalable approach for future use in complicated real-world environments.

1 Problem Statement and Contextualisation

In classical mythology, the labyrinth of Crete was navigated with *"Ariadne's thread,"* a lifeline provided by an outside observer to guide a vulnerable explorer safely through a complex, unknown maze. Modern autonomous robotics faces a similar problem. Unmanned Ground Vehicles (UGVs) used in search and rescue (SAR) or tactical situations typically rely on Global Positioning Systems (GPS) or active sensor emissions, such as LiDAR, to determine their location and plan navigable routes. [1]. However, in highly contested or unstructured environments, such as disaster zones, deep forest canopies, or electronic warfare scenarios, these signals can often be degraded, obstructed, or intentionally jammed [2].

When active signals fail, autonomous ground navigation systems become functionally blind. To address this critical vulnerability, this project presents **Ariadne**: an autonomous **air-to-ground optical routing pipeline**. This system acts as a modern digital thread, utilising a single high-altitude RGB image captured by an Unmanned Aerial Vehicle (UAV). It passively maps the environment below, identifies navigable corridors, and calculates the optimal path for the ground asset.

To test the core logic of the computer vision and pathfinding modules without the interference of unpredictable lighting conditions or organic textures found in the real world, this study uses procedurally generated mazes as a synthetic proof-of-concept. The maze environment acts as a controlled sandbox to isolate and evaluate the core algorithmic pipeline before scaling up to more complex, unstructured terrains.

1.1 System Requirements

To successfully achieve optical-based autonomous routing, the proposed pipeline must fulfil the following sequential requirements:

1. **Target Extraction:** The system must accurately detect and extract the pixel coordinates of the starting agent (the UGV) and the target destination (a human or objective) from the overhead image.

2. **Environmental Segmentation:** The system must classify the environment, accurately dividing the image into strictly traversable (pathways) and non-traversable (walls/obstacles) terrain.

3. **Tactical Pathfinding:** The system must compute an optimal, collision-free route from the start node to the target node using the segmented map, and project this route back onto the original visual space.

2 Literature Review

The difficulty of autonomous navigation in environments without GPS has led to significant research into alternative localisation and mapping methods.

Recent research on Unmanned Ground Vehicles (UGVs) has extensively explored the use of environment-specific, non-visual sensors to overcome the limitations posed by degraded optical conditions. The project applied **magnetic field map-based localisation**, which uses natural magnetic anomalies as navigational features [1]. This approach employs Gaussian Process (GP) Regression to create high-resolution magnetic signature maps. It also combines a hybrid Particle Filter (PF) and Unscented Kalman Filter (UKF) to estimate the trajectory of UGVs. In tests conducted in indoor corridors, this methodology demonstrated remarkable resilience, reducing localisation errors by as much as 45% compared to standard particle filter (PF)-based methods. However, this approach relies entirely on localised, ground-level sensing, meaning that the UGV must physically navigate and explore the environment to use the map effectively. **Ariadne** addresses this critical gap by changing the approach from *"ground-level discovery"* to *"overhead deterministic routing,"* enabling the UGV to receive an optimal path before encountering any obstacles.

Beyond ground-level localisation, extensive research has examined how Unmanned Aerial Vehicles (UAVs) navigate in GPS-denied environments. A prominent framework proposed for UAV exploration combines Simultaneous Localisation and Mapping (**SLAM**) with Partially Observable Markov Decision Processes (**POMDP**) [2]. In this model, navigation is treated as a sequential decision problem under uncertainty. Simulation results showed that the UAV could safely navigate, avoid collisions, and maximise information gain while creating an occupancy map of an unknown environment. While effective for a single agent, this framework focuses solely on the UAV's self-preservation and exploration. It does not consider the potential of utilising the generated occupancy map to actively guide a separate ground asset. **Ariadne** bridges this gap by using the UAV as an overhead optical *"oracle"* that segments the terrain to guide the movements of a UGV.

To achieve optical terrain mapping, deep learning-based semantic segmentation has become the standard. Recent advancements in aerial road extraction utilise Convolutional Neural Networks (CNNs), particularly the **U-Net architecture**, to carry out binary semantic segmentation on remote sensing imagery [3]. Research by Doghmane and Boukari demonstrates that enhancing the U-Net model with **Batch Group Normalisation** (BGN) significantly improves its ability to classify individual pixels into distinct object categories, such as road versus non-road areas. By dividing high-resolution aerial images into manageable patches and applying a sigmoid activation function to output a continuous pixel value, their BGN-UNet model achieved an exceptional Mean Intersection over Union (IoU) of 98.4%. **Ariadne** takes direct inspiration from this pixel-wise classification approach. Just as BGN-UNet isolates navigable infrastructure from complex backgrounds, *Ariadne* utilises deep learning segmentation on overhead UAV imagery to divide simulated tactical environments (represented via procedural mazes) into strictly traversable paths and non-traversable boundaries. However, while current literature primarily treats the segmented image as the final output, *Ariadne* expands on this idea by utilising the high-IoU segmentation mask as a foundational grid for A^* **pathfinding**. This approach translates the visual classification into a mathematical route for a ground agent.

3 Design and Methodology

The system architecture of *Ariadne* consists of four sequential phases: synthetic data generation, target extraction, environmental segmentation, and tactical pathfinding. This pipeline was implemented using a combination of Blender 5.1 for procedural environment modelling, and PyTorch (U-Net) and Ultralytics (YOLO) for computer vision processing.

3.1 Synthetic Data Generation

The basis of this proof-of-concept is to create a highly controlled and noiseless environment for the independent evaluation of the algorithmic logic in the computer vision and pathfinding modules. Before deploying optical routing architectures into unstructured, highly occluded real-world topographies (such as forest canopies or disaster zones), the core logic must be validated.

To achieve this, a custom dataset of 1000 procedural 2D mazes was generated using the Blender Python API ($\texttt{bpy}$). The dataset was partitioned into 800 mazes for training, 100 for validation, and 100 for testing. Utilising a 3D rendering engine (Figure 1) for top-down 2D maze generation enables automated extraction of mathematically perfect ground-truth labels.

For each procedural iteration, the rendering pipeline was configured to automatically export three distinct assets:

1. **RGB Overhead Image:** A colour rendering (Figure 2a) simulating the raw optical feed captured by the UAV's downward-facing camera.
2. **Binary Segmentation Mask:** A grayscale image (Figure 2b) mapping the precise pixel coordinates of paths (white) against walls (black). This serves as the ground truth for the U-Net architecture.
3. **Bounding Box Annotations:** A text file (Listing 1) containing the normalised spatial coordinates and class identifiers for the starting agent (0: UGV) and the target objective (1: Human). These labels adhere to the YOLO formatting standard, providing the ground truth required for the target extraction phase.

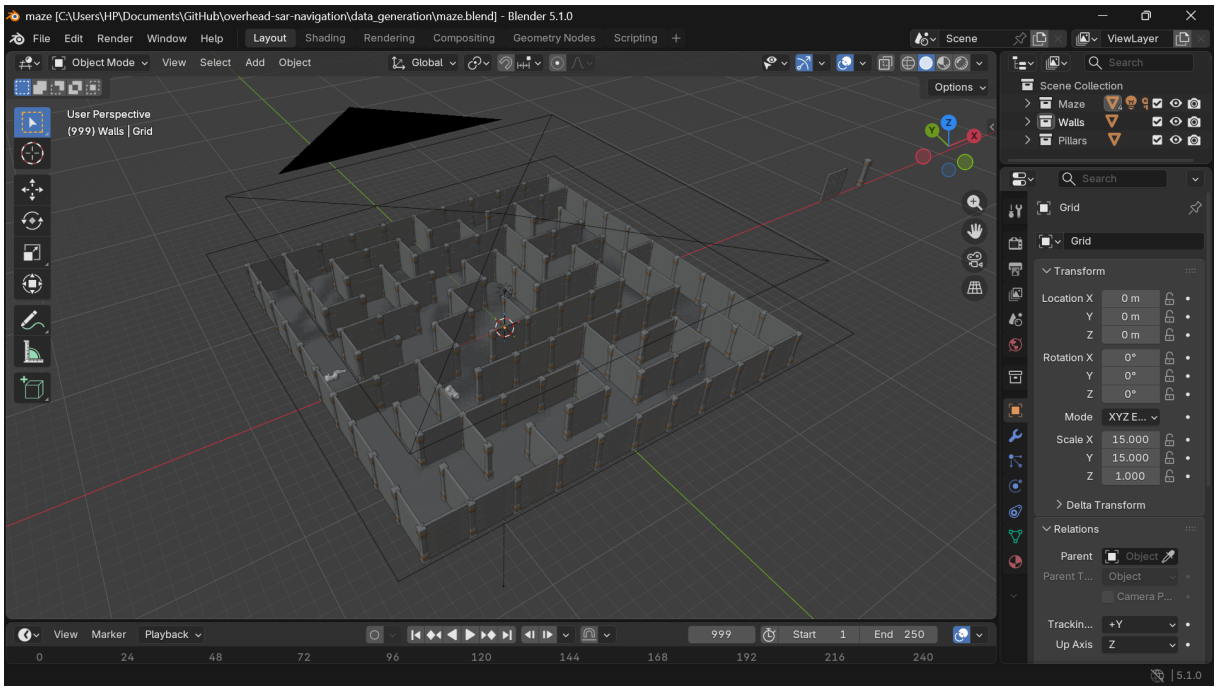
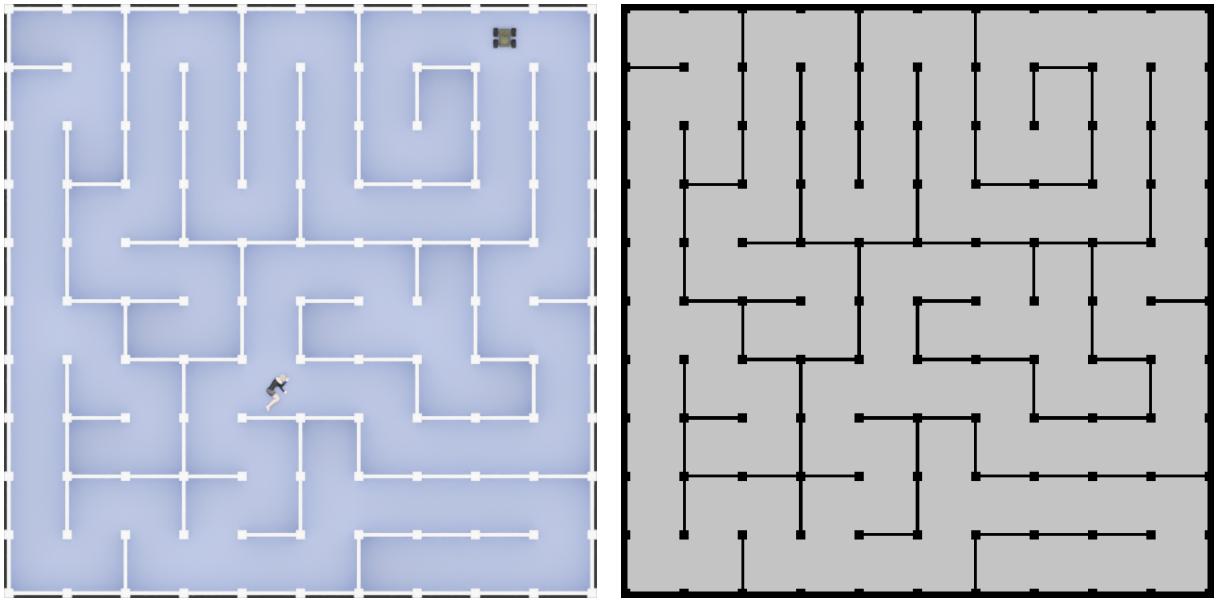


Figure 1: Procedural maze generation environment configured within Blender 5.1.



(a) RGB Overhead Image (b) Binary Segmentation Mask

Figure 2: Procedurally generated 2D maze renders

```
0 0.844262 0.057377 0.039220 0.034991
1 0.457275 0.653128 0.068142 0.071114
```

Listing 1: Normalised bounding box coordinate output

3.2 Target Extraction

Before an autonomous route can be calculated, the system must first determine the spatial coordinates of the start and end points within the unknown environment. In a real-world search and rescue (SAR) or tactical scenario, these points represent the Unmanned Ground Vehicle (the starting point) and the human target (the destination).

To achieve this, *Ariadne* uses a custom-trained object detection model built on the YOLO (You Only Look Once) framework. YOLO was chosen for this phase because of its advanced inference speed and high bounding box precision, which are essential for real-time tactical applications. The model is trained to analyse raw RGB overhead images captured by the UAV and to isolate the specific pixel clusters representing the UGV (Unmanned Ground Vehicle) and human subjects (Figure 3). The training process utilised the Ultralytics framework and was optimised for the specific resolution of the synthetic dataset (640×640px), as outlined in Listing 2.

```
# Train the model
model = YOLO('yolo26s.pt') # Load YOLO26 small model

# Save the results directly on Google Drive
gdrive_save_dir = '/content/gdrive/MyDrive/Colab Notebooks/overhead-sar-navigation/object_detection/runs'

results = model.train(
  data='/content/dataset.yaml',
  epochs=50,          # 50 complete passes through the 800 training images
  imgsz=640,          # Standard 640x640 resolution
  batch=16,           # Process 16 images simultaneously in VRAM
  device=0,           # Force PyTorch to utilize the T4 GPU
  project=gdrive_save_dir, # Auto-backs up weights to Google Drive
  name='maze_actor_detection_v1' # Name of the output subfolder
)

```

Listing 2: YOLO object detection model training configuration

3.2.1 Spatial Discretisation

After successfully drawing bounding boxes over the agents, the continuous visual space must be converted into a discrete mathematical format that is suitable for classical pathfinding algorithms. This spatial discretisation is inspired by the approach to aerial segmentation developed by Doghmane and Boukari [3]. In their research, high-resolution remote sensing imagery was systematically divided into fixed 256×256 spatial patches. This technique maximised computational efficiency while preserving important spatial relationships.

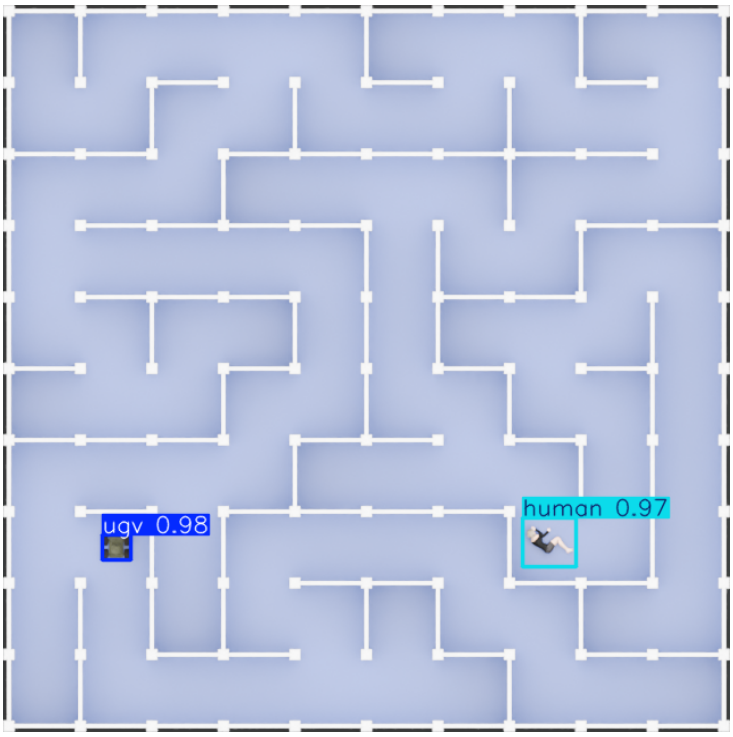


Figure 3: YOLO object detection model successfully extracting the UGV and human.

Ariadne applies this philosophy by downsampling the continuous optical feed into a grid defined by one main parameter (`grid_size`). From this parameter, the spatial constraints are mathematically derived:

1. **Image Size:** The raw pixel resolution of the square overhead optical input (e.g., 640×640).
2. **Grid Size:** The primary defined parameter establishing the matrix dimensions (e.g., 30×30).
3. **Cell Size:** A derived value calculated by dividing the `img_size` by the `grid_size`, representing the exact pixel span of a single grid cell.

To turn the YOLO bounding boxes into usable grid nodes, the extraction module finds the exact centre pixel of each bounding box. It then divides these pixel coordinates by the cell size and rounds them to whole numbers to find the specific (x, y) grid index where the agent is located. Finally, a bounding function ensures these values stay within the limits of the matrix array. This process is shown in Listing 3.

```python
# Calculate dimensions
px_w = x2 - x1
px_h = y2 - y1

# Calculate exact center pixel
px_x = x1 + (px_w // 2)
px_y = y1 + (px_h // 2)

# Convert pixel centers to grid indices for pathfinding
grid_x = int(px_x // self.cell_size)
grid_y = int(px_y // self.cell_size)

# Clamp values to ensure they never exceed grid bounds
grid_x = max(0, min(self.grid_size - 1, grid_x))
grid_y = max(0, min(self.grid_size - 1, grid_y))

```

Listing 3: Translating visual bounding boxes into bounded discrete grid nodes

3.2.2 Grid Resolution Tuning

The selection of the grid size parameter is a critical optimisation problem that directly dictates both the computational efficiency and the physical safety of the autonomous route.

Tuning the grid resolution requires balancing two competing extremes:

1. **Low Resolution (Under-sampling):** A smaller grid size significantly reduces the dimensionality of the search matrix, leading to exceptionally fast A^* execution times. However, if the grid resolution is too low, the system suffers from **spatial aliasing**. Thin boundary walls and traversable pathways are combined into the same discrete cell, leading the algorithm to falsely classify safe routes as blocked and frequently resulting in a "no route found" error.

2. **High Resolution (Over-sampling):** Conversely, a larger grid size heavily increases the complexity of the matrix, driving up the computational time required for the pathfinding module to solve the route. Furthermore, in a highly granular grid, the A^* algorithm's mathematical optimisation will inherently draw the shortest possible path, which often results in the route **"hugging" the absolute edges** of walls and corners. For a physical UGV, this lack of spatial tolerance introduces a severe risk of mechanical collision.

To achieve a balance between computational latency and navigational safety, *Ariadne* uses a proportional tuning strategy informed by the known dimensions of its procedural environment. Since the synthetic mazes are created on a basic 10×10 logical grid, the optical pathfinding matrix has been specifically adjusted to a 30×30 resolution.

This establishes a precise $1 : 9$ spatial mapping, where each cell of the original maze corresponds to a 3×3 block of cells in the pathfinding matrix. This specific ratio ensures that the thin boundary walls occupy separate grid nodes from the traversable corridors. It provides an adequate spatial buffer to prevent the pathfinder from clipping the edges of the walls, completely eliminates the risk of path occlusion, and maintains a sufficiently lightweight matrix for fast algorithmic execution.

3.3 Environmental Segmentation

Once the start and goal nodes are clearly defined, the next step in the pipeline is to classify the terrain in between. This involves dividing the optical feed into two categories: traversable paths and non-traversable boundaries, such as walls and obstacles. To assess the balance between computational latency and environmental robustness, this study develops and tests two different segmentation methods: a classical computer vision technique known as Colour Heuristics and a deep learning method called Neural Semantic Segmentation.

3.3.1 Approach A: Classical Colour Heuristics

The first approach uses classical, deterministic computer vision techniques using the OpenCV library. This module (`color_segmenter.py`) applies predefined colour thresholding arrays to accurately isolate traversable paths based solely on raw pixel values.

The primary advantage of this heuristic approach is its exceptional computational efficiency. Because it relies on simple array masking rather than passing tensors through neural layers, it executes in milliseconds. This makes it highly attractive for hardware with severe SWaP (Size, Weight, and Power) constraints.

However, this methodology is inherently brittle. Because the algorithm has no contextual understanding of the environment's topology, it is highly susceptible to stochastic noise. Minor variances in simulated lighting, shadows, or even the pixel anti-aliasing along the edges of the generated walls can cause the thresholding to fail, resulting in fragmented masks that trap the pathfinding algorithm.

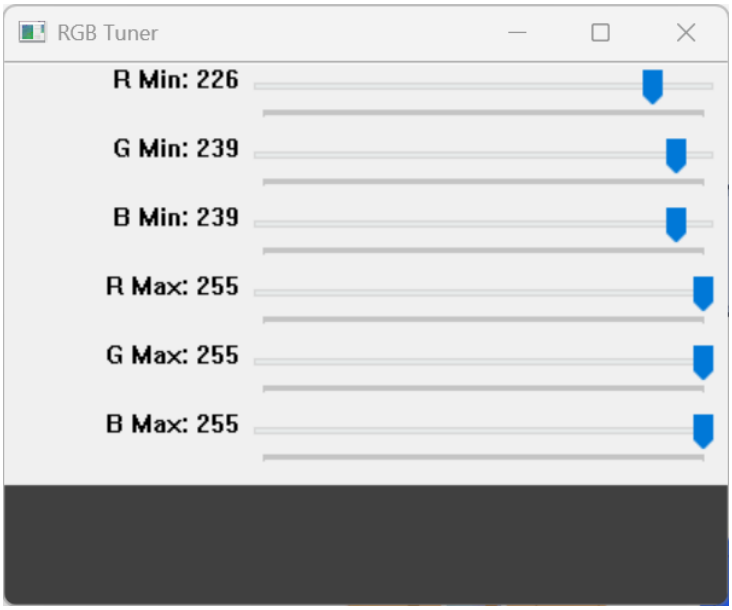
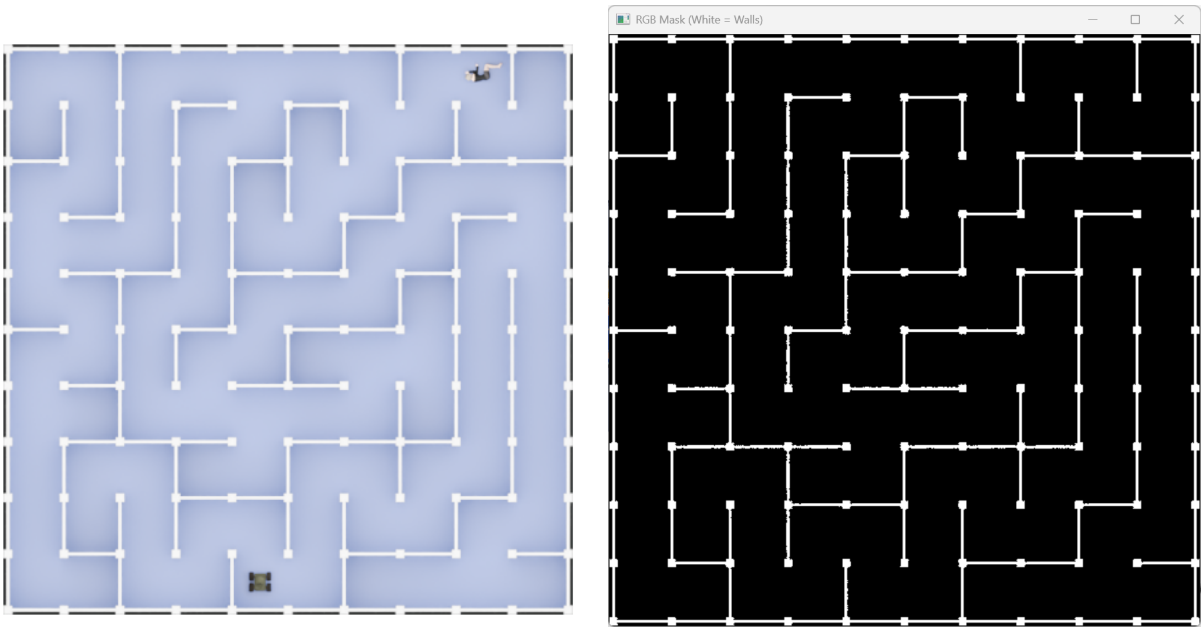


Figure 4: Interactive colour thresholding tuner used to set the optimal segmentation boundaries for Approach A.



(a) RGB Overhead Image (b) Colour Heuristic Mask

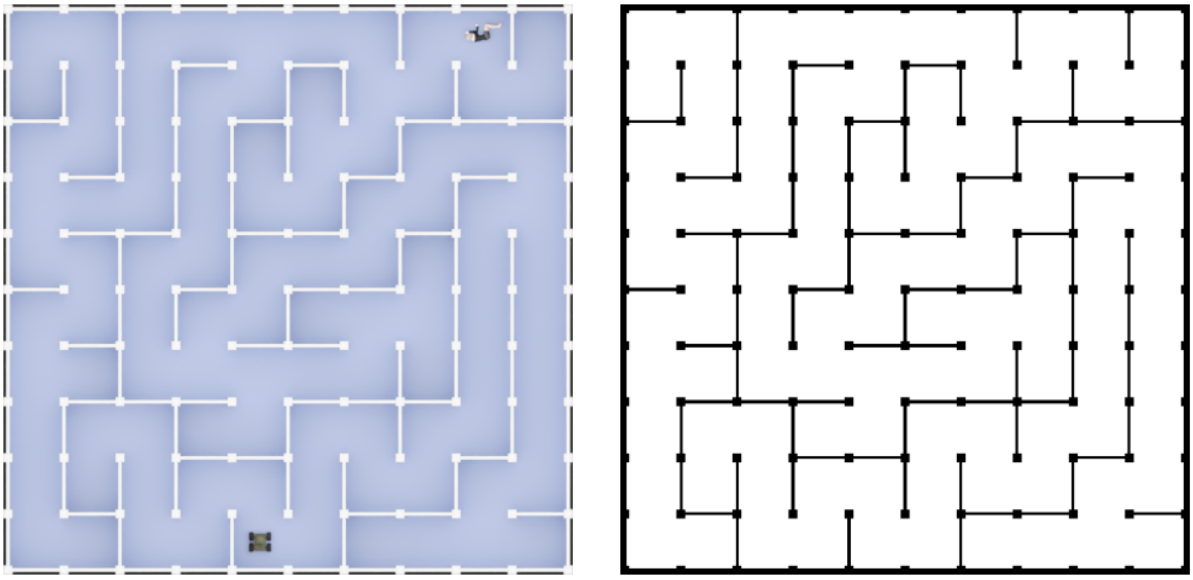
Figure 5: Environmental segmentation using colour thresholding. Note the susceptibility to anti-aliasing noise at the wall boundaries

3.3.2 Approach B: Neural Semantic Segmentation (U-Net)

To address the limitations of pure colour thresholding, the second approach uses a deep convolutional neural network (`unet_segmenter.py`). This method builds on architectural techniques discussed in the literature for extracting roads from aerial images [3]. It uses a PyTorch implementation of the U-Net architecture, enhanced by a pre-trained ResNet34 backbone to improve feature extraction.

Unlike the heuristic method, the U-Net model does not isolate pixels based on raw colour values. Instead, through its encoder-decoder structure, it learns the underlying spatial geometry and structural shape of the walls versus the corridors. This enables the model to produce highly accurate, continuous binary masks that are completely agnostic to lighting variations or texturing noise.

The model was trained using a cloud service (Google Colab) with the procedurally generated dataset. The inference execution is handled locally by the *Ariadne* pipeline to simulate edge deployment.



(a) RGB Overhead Image (b) Neural Semantic Mask

Figure 6: Environmental segmentation using the U-Net neural architecture, producing a robust and structurally aware binary mask.

3.3.3 Deployment Criteria

The choice to route optical data through the Colour Heuristic or the Neural Semantic module determines the overall effectiveness of the navigation system, which depends entirely on the operational environment.

- **When to deploy the Colour Heuristic:** This method is only practical in highly controlled environments that are synthetically perfect and have consistent lighting. Its application in modern robotics is restricted to small, weight-and-power-constrained (SWaP) edge devices (such as micro-drones) functioning in clean, predictable indoor settings where the computational demands of a neural network cannot be met.

- **When to deploy Neural Semantic (U-Net):** This is the required architecture for any transition to real-world, unstructured environments (such as disaster zones or forest canopies). While inference takes marginally longer and requires a GPU or localised AI accelerator (such as an NVIDIA Jetson), the U-Net's ability to maintain structural awareness through lighting shifts, partial occlusions, and environmental noise makes it the only viable solution to guarantee collision-free UGV routing.

3.4 Tactical Pathfinding

With the environmental grid successfully segmented into traversable and non-traversable nodes, the final phase of the *Ariadne* pipeline is to compute an optimal, collision-free route from the UGV's starting coordinate to the human target. To achieve this, the system uses the A^* (A-Star) algorithm.

3.4.1 Algorithm Justification and Movement Constraints

Modern robots often use sampling-based algorithms, like Rapidly-exploring Random Trees (RRT), to navigate complex movement paths. These algorithms work well in continuous, high-dimensional spaces. However, *Ariadne* simplifies the environment by turning it into a clear, two-dimensional grid. Because of this, a grid-based search method is more efficient and effective for finding paths.

To keep the ground agent safe, the algorithm only allows movement in four directions: up, down, left, and right. This restriction helps prevent the UGV from accidentally hitting tight corners. A^* algorithm was chosen instead of Dijkstra's Algorithm because it uses a spatial strategy that quickly narrows down the search area and moves directly toward the goal.

3.4.2 Heuristic and Cost Matrix Integration

The A^* algorithm works well because of a simple formula: $f(n) = g(n) + h(n)$. In this formula, $g(n)$ is the exact cost to get from the starting point to the current point n, while $h(n)$ is an estimate of the cost to get from point n to the goal.

To calculate $g(n)$, the pathfinder uses the binary segmentation mask created by the U-Net module. This mask acts as a map for obstacles: if a neighbouring cell has a pixel value of 0 (indicating a wall), the algorithm will not consider that node, making it impassable. Cells that can be traversed (with a pixel value of 1) have a step cost of 1.

To calculate the heuristic $h(n)$, the system uses the **Manhattan Distance**. Since the UGV can only move in four directions on a grid and not diagonally, the Manhattan Distance is a suitable choice. It is calculated as:

$$h(n) = |x_{target} - x_{current}| + |y_{target} - y_{current}|$$

This method is admissible and consistent, meaning it will never overestimate the true cost to reach the target. This code is shown in Listing 4.

```python
def __init__(self):
    # We restrict movement to 4 cardinal directions (Up, Down, Left, Right)
    # to prevent the UGV from clipping diagonally through wall corners.
    self.movements = [(0, 1), (0, -1), (1, 0), (-1, 0)]

def calc_manhattan_dist(self, current_node, target_node):
    """
    Calculates the grid-based distance between two points.
    It counts the strict horizontal and vertical steps required.
    """
    return abs(
        target_node[0] - current_node[0]) + abs(
        target_node[1] - current_node[1])

```

Listing 4: A^* movement constraints and Manhattan heuristic calculation

3.4.3 Route Output

The algorithm uses a priority queue from Python's `heapq` module to find the node with the lowest $f(n)$ score. When it successfully reaches the target node, it traces back the path of evaluated nodes to the starting node using a recorded history. This path is then reversed to create the best route as a list of (x, y) grid coordinates. In a live deployment, this list acts as the waypoint list sent to the UGV's flight controller, completing the Air-to-Ground optical navigation loop.

4 Results

The evaluation of the *Ariadne* pipeline looked at how well the system translated raw optical data into useful navigational waypoints. The tests took place in created, synthetic 2D mazes. Because the system could achieve close to 100% accuracy in a noise-free environment, focusing only on numbers does not offer much academic value. Instead, this section compares the system to the best current methods. It then evaluates how well the system meets its main Proof of Concept requirements.

4.1 Qualitative Literature Comparison

To evaluate the academic and tactical value of *Ariadne*, the architecture was compared against the three primary methodologies identified in the literature review: Magnetic Field Map-Based Localization [1], SLAM/POMDP Aerial Exploration [2], and BGN-UNet Aerial Road Segmentation [3]. The goal of this comparison, shown in Table 1, is to identify the strengths and weaknesses of each approach. This will help explain how *Ariadne* connects aerial perception with ground-level movement.

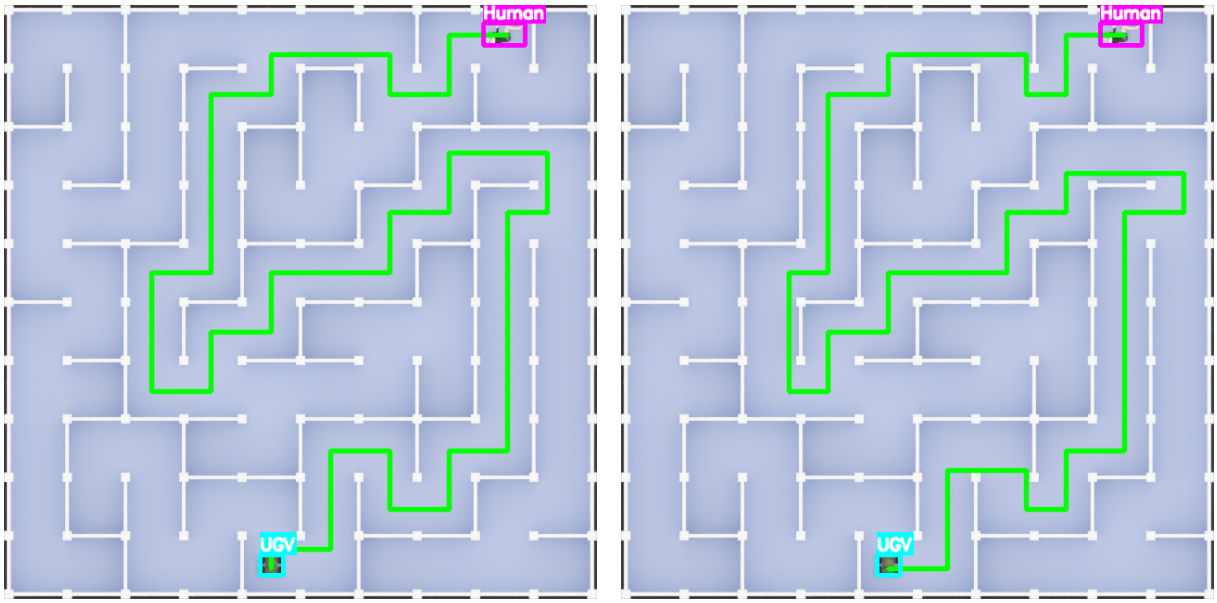
Methodology	Operational Domain	Key Strengths	Inherent Limitations
Magnetic Field Localization [1]	Ground-Level (UGV)	Highly resilient in visually degraded environments (e.g., thick smoke, darkness). Immune to GNSS jamming.	**Reactive Routing:** Requires "ground-level discovery." The UGV must physically traverse and encounter obstacles to map the environment.
SLAM & POMDP Framework [2]	Aerial-Level (UAV)	Excellent handling of sequential decision-making under uncertainty. Maximizes safe environmental exploration.	**Isolated Architecture:** Designed solely for single-agent self-preservation. Does not relay generated occupancy maps to separate ground assets.
BGN-UNet Segmentation [3]	Aerial-Level (UAV)	Exceptional pixel-wise classification of complex environments, achieving near-perfect (98.4%) geometric IoU.	**Static Output:** Treats the semantic mask as the final deliverable. Does not mathematically translate the classification into kinematic routes.
Ariadne (Proposed System)	**Cross-Domain** (UAV-to-UGV)	**Deterministic Routing:** Provides the UGV with an optimal path *before* physical movement. Bridges computer vision and classical pathfinding (A^*).	**PoC Constraints:** Currently relies on synthetic environments. Requires clear overhead optical visibility, lacking the sensor redundancy of magnetic or LiDAR systems.

Table 1: Qualitative Comparison of Autonomous Navigation Methodologies

Current research often separates aerial and ground operations, as shown in Table 1. *Ariadne* solves this problem. It uses the UAV as an overhead optical *"oracle"*, avoiding the risks associated with ground-level detection (seen in [1]). By sending the U-Net segmentation mask into an A^* search grid, *Ariadne* overcomes the limitations of static image outputs in [3]. This process allows it to turn pixels into a usable mathematical route.

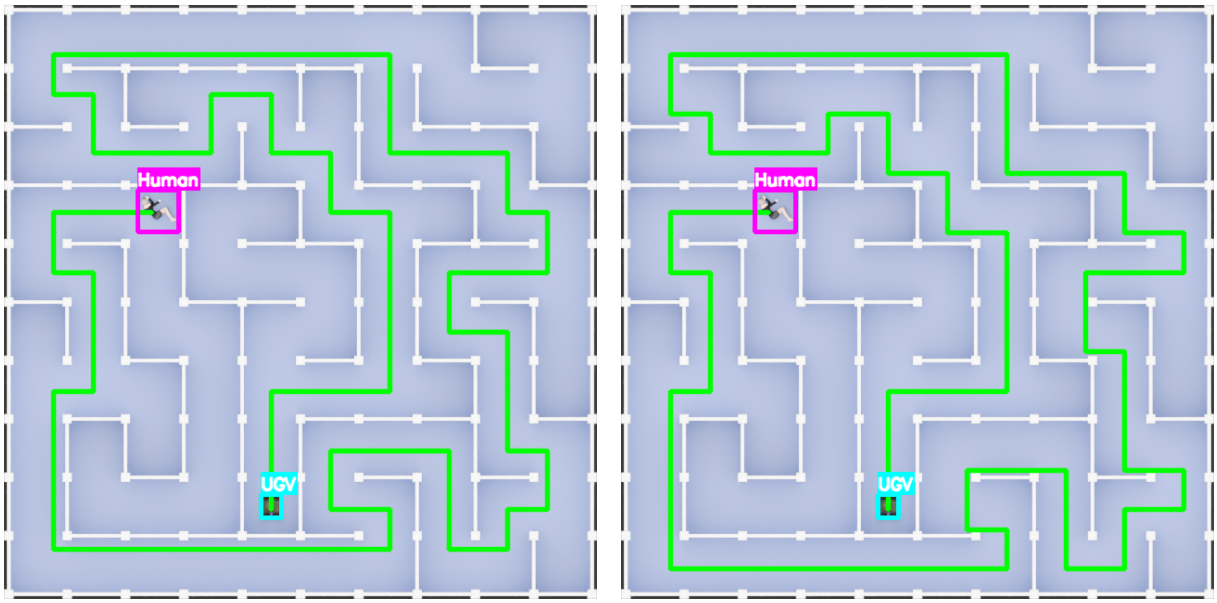
4.2 Visual Pathfinding Results

To validate the system's core logic, the pipeline was tested against 100 procedurally generated environments of varying spatial complexity. The results below show that both the classical colour heuristic and the neural semantic (U-Net) routing methods completed the tasks successfully.



(a) Colour Heuristic Solution (b) Neural Semantic Solution

Figure 7: Visual comparison of pathfinding solutions in a simpler maze. Both the colour heuristic (a) and the neural semantic approach (b) compute a direct, low-complexity route to the target.



(a) Colour Heuristic Solution (b) Neural Semantic Solution

Figure 8: Routing performance in a more complex maze. Both methods effectively find a complex route with multiple turns. This shows that the system can safely navigate through high-density obstacles.

4.3 Limitations and Scope

It is critical to contextualise these results within the scope of a synthetic Proof of Concept (PoC). The primary objective here was to prove the underlying mathematical logic of an Air-to-Ground optical pipeline, not to immediately deploy a production-ready model into a physically chaotic environment.

The procedural 2D mazes served as a controlled and noiseless sandbox. However, the current system has two main limitations. First, it assumes that the UAV can always see the UGV from above without any obstructions. In a real forest, thick trees can completely block the UAV's view. Second, the system has not been tested with random lighting, weather changes, or various natural textures found in the real world. Still, since the system is very modular, future updates can adjust to real-world conditions by retraining the U-Net and YOLO models using generated data similar to such scenarios.

5 Ethics, Application, and Robustness

5.1 Real-World Application

The *Ariadne* architecture is built to address a key challenge in modern robotics: navigating without GPS. It is mainly meant for Search and Rescue (SAR) missions in environments that are hard to navigate, such as disaster areas with collapsed buildings or floods. In these situations, ground sensors like LiDAR can struggle due to smoke or debris, and satellite navigation cannot be used. By using a high-altitude UAV as a visual guide, emergency responders can safely direct UGVs to reach victims without risking the safety of human operators.

5.2 Robustness and Consequences of Error

Ariadne is an optical routing system that relies on the quality of the U-Net segmentation mask. If this system is used in real-life situations, any errors in computer vision can lead to serious physical failures.

There are two primary failure modes:

1. **False Positives (Hallucinating a Path):** If the segmentation model falsely classifies an impassable obstacle (such as a ditch or a fallen tree) as traversable terrain, the A^* algorithm will confidently route the UGV directly into it, resulting in a mechanical collision, vehicle loss, or mission failure.

2. **False Negatives (Hallucinating a Wall):** If the model is confused by visual noise (such as shadows) and classifies a clear path as an obstacle, the A^* matrix may falsely conclude that the target is completely blocked, leaving the UGV stranded and unable to reach a critical objective.

To mitigate these consequences, a production-level deployment of *Ariadne* cannot operate in a vacuum. It must act as a *"Global Planner,"* generating the overarching route, while the UGV utilises localised edge sensors (such as bump sensors or short-range LiDAR) as a "Local Planner" to reactively avoid unmapped, immediate physical threats.

5.3 Ethics and the Dual-Use Dilemma

This project was created for humanitarian Search and Rescue efforts, but any technology that allows for autonomous navigation in areas without GPS can also be used in other ways. The same systems that guide a medical supply drone through a chaotic disaster zone can be used by military or other groups to direct dangerous weapons through an electronic warfare environment without needing a human to oversee the process.

This raises serious ethical questions about who is responsible. If an autonomous ground vehicle using an *Ariadne*-style optical system mistakes a target or goes through a protected civilian area, it becomes unclear who is accountable. Taking the human operator out of the navigation process means that strict rules must follow international humanitarian law. Future developers need to create reliable safety measures to make sure that the system cannot be used as a weapon without human permission.

6 Conclusion

The success of the *Ariadne* pipeline shows that overhead optical routing is effective in areas without GPS. It combines target extraction, AI-driven segmentation, and pathfinding into one automated system, linking what a drone sees with how a ground vehicle moves. One of the most exciting advancements of this system is how it reimagines deep learning outputs. Instead of seeing a segmentation mask as just a picture, *Ariadne* turns it into a cost map, transforming raw pixels into rules for navigation.

This logic opens up the possibility for better environmental mapping. Instead of limiting the options to just passable paths or impassable walls, future versions can identify different types of terrain, like paved roads, thick mud, or shallow water. By giving each type of terrain a different movement weight, the A^* pathfinder can create routes that are not only the shortest but also the safest and most energy-efficient for the hardware to use.

References

[1] B. Dikic, I. Okanovic, S. Grubmüller, S. Solmaz, H. Sikic, and Q. Cao, "Magnetic field map-based localization: A novel approach for search and rescue robotics," in *2025 IEEE International Automated Vehicle Validation Conference (IAVVC)*, pp. 1–6, 2025.

[2] F. Vanegas, K. J. Gaston, J. Roberts, and F. Gonzalez, "A framework for uav navigation and exploration in gps-denied environments," in *2019 IEEE Aerospace Conference*, pp. 1–6, 2019.

[3] R. Doghmane and K. Boukari, "Enhanced u-net model for accurate aerial road segmentation," *Machine Graphics & Vision*, vol. 33, p. 71–96, Dec. 2024.